

MODULE 32

Resilience and Fault Tolerance Patterns

Learn how to build resilient, fault-tolerant microservices with Resilience4j that stay reliable even when failures happen.





WHY DISTRIBUTED SYSTEMS FAIL

In a distributed environment, failures are not just possible -- they are inevitable.

THE REALITY OF DISTRIBUTED SYSTEMS

Unlike a single-server application, distributed systems consist of many independent components that run across networks and infrastructure beyond our direct control. They communicate over network calls, share data asynchronously, and depend on external services -- any of these dependencies can fail at any time.

COMMON REASONS WHY DISTRIBUTED SYSTEMS FAIL

Reason	Description
Network Issues	Latency, packet loss, DNS failures, or intermittent connectivity break service communication.
Component Failures	Services, databases, or instances can crash, become unresponsive, or run out of resources.
Timeouts	Slow responses or hanging calls cause timeouts and cascading failures across services.
External Dependency Failures	Third-party APIs, external services, or cloud resources may become unavailable or rate-limited.
Data Inconsistency	Partial updates, replication delays, or split-brain scenarios can lead to inconsistent data.
Overload Conditions	Traffic spikes or insufficient capacity can overwhelm services and degrade performance.
Configuration & Human Errors	Wrong configurations, bad deployments, or manual mistakes can introduce outages.
Infrastructure Problems	Disk failures, zone outages, or power issues can impact multiple services simultaneously.

KEY TAKEAWAY Failures are normal in distributed systems. The goal is not to prevent failures, but to detect them early, contain the impact, and recover quickly.



MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to design and implement resilient, fault-tolerant microservices that remain reliable under failure.

MODULE FORMAT

Lecture + Lab

Theory: 1 Hour

Lab: 2 Hours

Total: 3 Hours

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO:

#	Objective	Details
01	Understand System Failures	Explain why distributed systems fail and identify common failure scenarios.
02	Apply Retry Patterns	Implement retry mechanisms using fixed retry and exponential backoff strategies.
03	Implement Circuit Breakers	Design and configure circuit breakers and understand the Closed, Open, and Half-Open states.
04	Use Timeouts and Fallbacks	Configure timeouts and implement fallback strategies to ensure graceful degradation.
05	Leverage Resilience4j	Use Resilience4j to implement retries, circuit breakers, and other resilience patterns in Spring Boot.
06	Build Highly Available Systems	Apply enterprise resilience principles and best practices to build highly available, fault-tolerant microservices.

KEY TAKEAWAY Ultimate goal: design and build microservices that can withstand failures, recover quickly, and continue delivering a great experience to your users.

UNDERSTANDING SYSTEM FAILURES

Failures are a natural part of distributed systems. Understanding how and why they occur is the first step to building resilient applications.

TYPES OF SYSTEM FAILURES

Type	What It Means (with Example)
Transient Failures	Temporary issues that resolve on their own. Example: a brief network glitch or momentary service unavailability.
Partial Failures	Only part of the system is affected. Example: one instance is down while others succeed.
Intermittent Failures	Failures that come and go, making them hard to reproduce and diagnose. Example: a flaky third-party API.
Persistent Failures	Long-lasting issues requiring manual intervention. Example: a database outage, misconfiguration, or resource exhaustion.
Cascading Failures	A failure in one component triggers failures in others, amplifying the impact. Example: service timeout -> retries -> overload -> outage.

WHY DO SYSTEMS FAIL?

- Network -- latency, packet loss, DNS issues.
- Infrastructure -- disk, zone, power outages.
- Application -- bugs, resource leaks, bad deploys.
- Data -- inconsistency, corruption, replication lag.

IMPACT OF UNHANDLED FAILURES

- Service downtime and loss of availability.
- Poor user experience and lost trust.
- Data inconsistency or loss.
- Business and financial impact.

KEY TAKEAWAY Failures are inevitable. Resilience is a choice. Understand the types, sources, and impact of failures so you can design systems that detect, isolate, and recover quickly.



COMMON FAILURE SCENARIOS

Failures can occur at different layers of a distributed system. Knowing the common scenarios helps us design the right resilience strategies.



Network Failures

Connectivity problems prevent services from communicating. E.g. high latency, packet loss, DNS resolution failure.



Service Unavailability

A service or instance is down or not reachable. E.g. service crash, instance terminated, health check failures.



Timeouts

A service takes too long to respond, causing the request to time out. E.g. slow queries, long external API calls.



Resource Exhaustion

A service runs out of essential resources. E.g. CPU saturation, memory leaks, disk full, connection pool exhausted.



Dependency Failures

An external dependency (service, API, database) fails or becomes slow. E.g. third-party API down, cache failure.



Data Inconsistency

Data state becomes inconsistent due to partial updates or concurrent operations. E.g. replication lag, stale reads.



Configuration Errors

Incorrect or missing configuration leads to component failures. E.g. wrong endpoint, invalid credentials.



Traffic Spikes

A sudden increase in traffic overwhelms the system. E.g. flash sales, viral promotions, improper rate limiting.

KEY TAKEAWAY Failures are inevitable in distributed systems. By understanding these common scenarios, we can design systems that detect, isolate, and recover quickly.

DESIGNING FOR FAILURE (1 OF 2)

In distributed systems, failures are inevitable. The goal is not to prevent failures, but to design systems that can anticipate, withstand, and recover from them.

CORE PRINCIPLES

- **Expect Failure** — assume that any component can fail at any time.
- **Isolate Failure** — limit the blast radius so failures don't spread to the whole system.
- **Recover Quickly** — enable automatic recovery and graceful handling of failures.
- **Degrade Gracefully** — provide limited functionality instead of complete failure.
- **Observe Continuously** — monitor, detect, and gain visibility to act before users are impacted.

KEY DESIGN PRACTICES

Practice	What It Means
Redundancy	Remove single points of failure with replicas and multiple zones.
Loose Coupling	Design services with clear boundaries and minimal dependencies.
Async Communication	Use messaging to decouple services and absorb spikes.
Retries with Control	Retry transient failures with backoff and jitter to avoid overload.
Circuit Breakers	Stop calling failing services temporarily to allow recovery.
Timeouts	Set time limits for calls to prevent threads waiting indefinitely.
Fallbacks	Provide alternative responses or default behavior when needed.
Idempotency	Design operations to be safe to retry without duplicate effects.
Health Checks	Continuously verify the health of services and dependencies.

KEY TAKEAWAY Resilient systems are designed with failure in mind. Anticipate it. Isolate it. Recover from it. Keep delivering value.

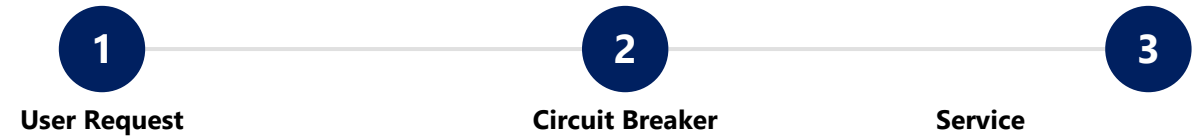
DESIGNING FOR FAILURE (2 OF 2)

A resilient design mindset, paired with a concrete example of these principles working together in a real request flow.

THE RESILIENT DESIGN MINDSET

- Build for resilience, not just for the happy path.
- Test for failure as much as you test for success.
- Embrace chaos in non-production environments.
- Continuously improve based on real-world incidents.

REAL-WORLD EXAMPLE: CIRCUIT BREAKER + FALLBACK



On success, the circuit breaker forwards the request and the service responds normally. On failure or timeout, the circuit breaker returns a Fallback Response instead of waiting or crashing.

WHY IT WORKS

When the service fails, the circuit breaker opens and a fallback response is returned to the user, ensuring a better experience than a hang or a crash.

KEY TAKEAWAY Resilient systems are designed with failure in mind: anticipate, isolate, recover, and keep delivering value -- even in a degraded state.

TRY IT YOURSELF — EXERCISE 1: WHY RESILIENCE

Reinforces: why an unprotected Account Profile call threatens the whole CRM request -- an analysis exercise.

THE SCENARIO

Customer detail for CUS-1001 (Amina Khan) calls the Account Profile service. The dependency hangs for 30 seconds -- and nothing in the CRM's code today bounds that wait, retries it, or protects the rest of the system while it happens.

STEPS (IN `notes/lab32-resilience.md`)

Step	What To Do
1 -- Hang Effects	List three user-visible or thread-pool effects of the 30-second hang (e.g. the customer detail page spins forever, the request thread is blocked and unavailable for other customers, health checks may start failing).
2 -- Name The Patterns	Write the four Resilience4j ideas this module teaches: Retry, Circuit Breaker, Time Limiter, Fallback.
3 -- Know The Limits	Write one sentence: resilience wraps calls -- it does not fix a permanently wrong URL or a broken configuration.
4 -- Save Your Work	Save the scenario analysis and pattern list under <code>notes/lab32-resilience.md</code> .

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: `exercises/exercise-01-why-resilience.md` (workspace: `examples/module-32-exercises`).

RETRY PATTERN OVERVIEW

Retry patterns automatically re-attempt a failed operation that is likely to succeed when the failure is transient.

WHAT IS A RETRY PATTERN?

A retry pattern makes multiple attempts to complete an operation that has failed due to a temporary issue, such as network glitches, timeouts, or service unavailability.

Key idea: keep trying with control and intelligence, not blindly and indefinitely.

WHEN TO USE RETRY

- Transient failures (timeouts, network issues).
- Intermittent service unavailability.
- Rate limiting (HTTP 429).
- Optimistic concurrency conflicts.
- Short-lived infrastructure glitches.

Do NOT retry permanent failures: validation errors, 4xx client errors (except 429), invalid requests.

BENEFITS

- **Improved Reliability** — handles temporary issues automatically.
- **Better UX** — requests succeed without user intervention.
- **Resilience** — recover from short-lived failures gracefully.
- **Reduced Manual Effort** — less need for human monitoring and re-tries.

KEY CONSIDERATIONS

Retry only what is safe

Limit the number of retries

Add delay between retries (backoff)

Add randomness (jitter)

Monitor and tune

KEY TAKEAWAY Use retry patterns wisely to handle transient failures and improve reliability, but always combine them with other resilience patterns like timeouts, circuit breakers, and fallbacks.

RETRY STRATEGIES: FIXED VS EXPONENTIAL BACKOFF

Two common retry timing strategies -- a constant delay between attempts, or a delay that grows with each attempt.

FIXED RETRY -- CONSTANT DELAY

Retries a failed operation a fixed number of times with the SAME delay between each attempt. Simple, predictable, and good for brief transient glitches.

Attempt	Result	Next Action
1 (t=0s)	Fails	Wait 1s, retry
2 (t=1s)	Fails	Wait 1s, retry
3 (t=2s)	Fails	Wait 1s, retry
4 (t=3s)	Succeeds	Return result

maxRetries = 3, delay = 1s. If attempt 4 also failed, the caller would give up and return the error.

EXPONENTIAL BACKOFF -- INCREASING DELAY

Delay = baseDelay x 2^(attempt-1). Each retry waits longer than the last, reducing load and giving the system more time to recover. Add jitter to avoid a thundering herd.

Attempt	Result	Next Wait
1 (t=0s)	Fails	Wait 1s (2^0)
2 (t=1s)	Fails	Wait 2s (2^1)
3 (t=3s)	Fails	Wait 4s (2^2)
4 (t=7s)	Fails	Wait 8s (2^3)
5 (t=15s)	Succeeds	Return result

baseDelay = 1s, multiplier = 2, maxAttempts = 5. Total wait before the last attempt = 1+2+4+8 = 15 seconds.

KEY TAKEAWAY Fixed retry is simple and predictable for short-lived glitches; exponential backoff (with jitter) scales the delay to reduce load and improve recovery odds under heavier or longer outages.



WHAT IS A CIRCUIT BREAKER?

The Circuit Breaker pattern prevents a failing service from causing cascading failures by stopping requests to it for a period of time, allowing it to recover.

WHAT IS A CIRCUIT BREAKER?

A circuit breaker monitors calls to a service. When failures exceed a threshold, it 'opens the circuit' and blocks further calls for a configured time. After the timeout, it allows a few test calls to check if the service has recovered.

REAL-WORLD ANALOGY: ELECTRICAL CIRCUIT BREAKER



Goal: fail fast, reduce latency, and protect your system and its dependencies.

KEY BENEFITS

Prevents Cascading Failures

Improves System Resilience

Reduces Latency & Waste

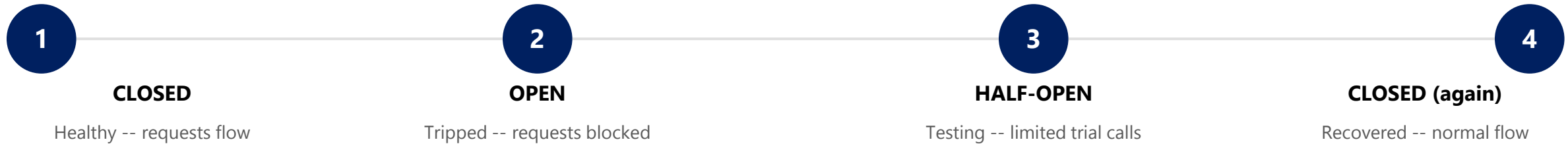
Better User Experience

Configurable & Observable

KEY TAKEAWAY A circuit breaker acts like a smart gatekeeper: it blocks traffic to a failing service, gives it time to heal, and only lets traffic through again when it's healthy. Remember: it's not about avoiding failures, it's about managing their impact.

CIRCUIT BREAKER STATES

A Circuit Breaker moves between three states -- Closed, Open, and Half-Open -- based on failures and timeouts, to protect your system and allow recovery.



Closed -> Open when the failure threshold is exceeded within the sliding window. Open -> Half-Open after the wait duration elapses. Half-Open -> Closed if test calls succeed; Half-Open -> Open again if any test call fails.

TYPICAL CONFIGURATION

Failure Rate Threshold: e.g. 50%

Minimum Number of Calls: e.g. 10

Sliding Window: last 30s

Wait Duration in Open: e.g. 30s

Permitted Calls in Half-Open: e.g. 5

EXAMPLE SCENARIO: PAYMENT SERVICE IS DOWN

Circuit opens after 10 failures in 30s (threshold hit) -> all further calls fail fast for 30s -> after 30s, 5 test calls are allowed -> if they succeed the circuit closes; if they fail it opens again for another 30s.

KEY TAKEAWAY Circuit breaker states work together to detect failures early, stop the bleeding, and then cautiously restore traffic. Fail fast, stay resilient, recover smart.

CIRCUIT BREAKER STATE DETAILS

Closed, Open, and Half-Open each have distinct behavior, purpose, and a trigger for moving to the next state.

State	Behavior	Purpose	Moves To...
CLOSED (Healthy)	All requests pass through and are forwarded to the service. Successes and failures are both counted; successful calls do NOT reset the failure count.	Allow traffic while the service is believed healthy, while continuously watching for trouble.	OPEN, if failures exceed the threshold within the sliding window.
OPEN (Tripped)	All incoming requests fail immediately without ever calling the service -- they fail fast.	Prevent overload and give the failing service time to recover without added load.	HALF-OPEN, once the configured wait duration has elapsed.
HALF-OPEN (Testing)	Only a limited number of test calls are allowed through; any additional requests during this window are rejected.	Safely probe the dependency to verify recovery without risking full traffic.	CLOSED if all test calls succeed; back to OPEN (with a fresh wait duration) if any test call fails.

WORKED EXAMPLE: PAYMENT SERVICE

Payment Service is up: the circuit stays Closed while occasional failures are tracked below the threshold. Ten failures occur within 30 seconds, crossing the threshold -- the circuit Opens, and every request fails fast for 30 seconds. After 30 seconds, the circuit moves to Half-Open and allows 5 test requests: if they all succeed, the circuit Closes and normal traffic resumes; if any fails, it reopens for another 30 seconds.

KEY TAKEAWAY Closed is the default healthy state; Open is pure protection; Half-Open is a careful, limited probe. Succeed -> close and heal. Fail -> open again and protect.

TRY IT YOURSELF — EXERCISE 2: CIRCUIT STATES

Reinforces: documenting Closed, Open, and Half-Open for the Account Profile breaker -- a documentation exercise.

STATE NOTES (IN notes/circuit-states.md)

State	What To Write
Closed	Normal calls flow to Account Profile; both successes and failures are counted.
Open	Calls fail fast or use the fallback; Account Profile is not hammered while it's unhealthy.
Half-Open	A small number of trial calls probe recovery; success -> Closed, failure -> Open again.

DRAW THE STATE DIAGRAM (BOXES + ARROWS)



KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-circuit-states.md (mention the fallback explicitly for the Open state).

TIMEOUT MANAGEMENT

Timeouts ensure that slow or unresponsive services don't block resources and degrade system performance. They define the maximum time to wait for a response.

WHY TIMEOUTS MATTER

- **Prevent Resource Exhaustion** — threads waiting too long can exhaust pools and degrade the entire system.
- **Improve Responsiveness** — fail fast when a service is slow, so the system stays responsive.
- **Enable Other Patterns** — timeouts are essential for retries, circuit breakers, and fallbacks to work.
- **Better UX** — users get a quick failure or fallback instead of a long, silent wait.

TIMEOUT TYPES

- **Connection Timeout** — maximum time to establish a connection. Example: 2 seconds.
- **Read Timeout** — maximum time to wait for a response after the request is sent. Example: 3 seconds.
- **Overall Timeout** — maximum total time for connect + send + wait. Example: 5 seconds.

TimeLimiter (application.yml)

```
resilience4j:  
  timelimiter:  
    instances:  
      accountProfile:  
        timeout-duration: 1500ms  
        cancel-running-future: true
```

TIMEOUT IN ACTION

If the response is not received within the configured timeout, the call is aborted (cancelled) and control returns immediately -- to retry, fall back, or fail fast -- instead of hanging.

KEY TAKEAWAY Timeouts protect your system by ensuring no call waits forever. Set the right values, monitor them, and combine with retries and circuit breakers for true resilience.

FALLBACK STRATEGIES

Fallback strategies provide an alternative way to deliver value when the primary service is unavailable, slow, or returns an error.

WHY FALLBACKS MATTER

- **Business Continuity** — keeps the application functional even when external services fail.
- **Better UX** — users get a meaningful response instead of errors or long waits.
- **Reduced Load** — serve cached or default data quickly instead of waiting on a failed call.
- **Protects Downstream** — avoids cascading failures by providing a safe alternative.

COMMON FALLBACK STRATEGIES

- **Cached Response** — return data from cache when the primary service fails.
- **Default / Static** — return a predefined default value or message.
- **Stale Data** — return the last known good data, even if outdated.
- **Alternate Service** — use a different service or provider to fulfill the request.
- **Graceful Degradation** — return a partial response with limited functionality.

AccountService.java

```
@CircuitBreaker(name = "accountProfile",
    fallbackMethod = "fallback")
public CompletableFuture<AccountSummary>
    find(String customerId) {
    return client.fetch(customerId);
}

private CompletableFuture<AccountSummary>
    fallback(String customerId, Throwable ex) {
    log.warn("account_profile_degraded");
    return CompletableFuture.completedFuture(
        AccountSummary.unavailable(customerId));
}
```

Fallbacks should be fast, reliable, and provide useful information -- even if it's degraded or partial.

KEY TAKEAWAY Fallbacks keep your application running and your users happy when things go wrong. Always provide a useful alternative, monitor it, and keep it simple.

GRACEFUL DEGRADATION

Graceful degradation keeps a service usable even when a dependency fails -- instead of breaking the whole request, return fallback or simplified data.

HOW IT WORKS



When a non-critical dependency fails, show cached or partial data and keep the core application usable -- never show a broken interface or a raw error for something the user could otherwise still do.

WHEN TO USE

- External APIs unavailable
- Non-critical components fail
- Data is stale or incomplete
- Maintain core functionality

GRACEFUL DEGRADATION VS. TOTAL FAILURE

Graceful Degradation	Total Failure
Something works (partially).	Nothing works.
Users can still continue.	Users are blocked.
Better UX and retention.	High frustration.

KEY TAKEAWAY Graceful degradation ensures your application remains useful and reliable -- even when a downstream dependency goes wrong.

TRY IT YOURSELF — EXERCISE 3: FALLBACK CONTRACT

Reinforces: specifying an honest, minimal Account Profile response for Amina when the dependency fails -- a documentation exercise.

DEGRADED CONTRACT (IN `notes/fallback-contract.md`)

Decision	What To Write
Fields Kept	customerId, displayName (if cached), status = UNKNOWN.
Fields Dropped	balance, tier, lastLogin -- anything that could be stale or wrong is omitted, not guessed.
API Signal	Choose HTTP 200 with degraded=true, or HTTP 503 -- write one sentence justifying your choice.
User Message	Draft one UI string, e.g. "Account details temporarily limited."

WHY THIS MATTERS

A fallback that looks like a normal success response is worse than an honest error -- Lab 32's own rubric treats a fallback that implies a successful write as an honor violation, not just a bug.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: `exercises/exercise-03-fallback-contract.md`.

INTRODUCTION TO RESILIENCE4J (1 OF 2)

Resilience4j is a lightweight, fault-tolerance library for Java that helps you build resilient applications with easy-to-use, composable components.

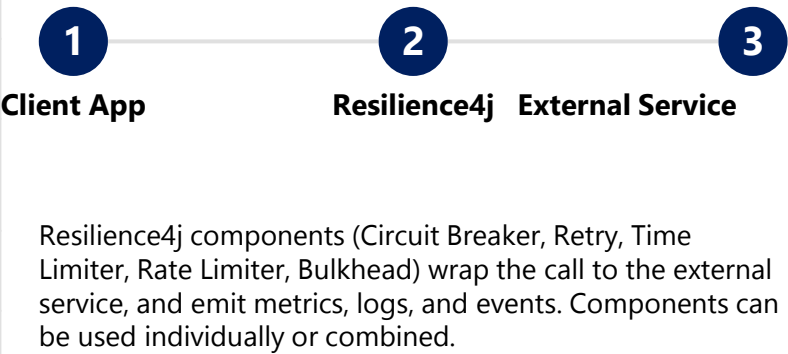
KEY FEATURES

- **Lightweight** — minimal dependencies and low overhead.
- **Modular** — choose only the resilience components you need.
- **Functional Style** — designed for Java 8+, functional interfaces, and lambdas.
- **Micrometer Compatible** — built-in metrics integrate with Prometheus, Grafana, etc.
- **Cloud-Native Ready** — perfect for microservices, reactive, and distributed systems.

CORE COMPONENTS

Component	What It Does
Circuit Breaker	Detects failures; blocks calls to a failing service until it recovers.
Retry	Retries a failed operation a configurable number of times before giving up.
Time Limiter	Limits execution time of a call and cancels it if it exceeds the timeout.
Rate Limiter	Controls the rate of calls to a service and prevents overload.
Bulkhead	Limits concurrent calls to isolate resources and prevent cascading failures.

HIGH-LEVEL ARCHITECTURE



KEY TAKEAWAY Resilience4j gives you the building blocks to make your services resilient, scalable, and production-ready. Use the right component for the right problem, and combine them for maximum resilience.

INTRODUCTION TO RESILIENCE4J (2 OF 2)

Why teams choose Resilience4j, a concrete worked example, and how it fits into the wider observability ecosystem.

WHY USE RESILIENCE4J?

- Improve system availability and reliability.
- Handle failures gracefully instead of crashing.
- Prevent cascading failures in distributed systems.
- Improve user experience with fast, honest failure.
- Gain visibility into failures with metrics and events.
- Highly customizable and production-ready.

EXAMPLE USE CASE: ORDER SERVICE -> PAYMENT SERVICE

- **Circuit Breaker** — prevents calls when Payment Service is down.
- **Retry** — attempts the call again for transient failures.
- **Time Limiter** — ensures the call does not hang indefinitely.
- **Rate Limiter** — protects Payment Service from too many requests.
- **Bulkhead** — isolates Payment calls from other operations.

ECOSYSTEM INTEGRATION

Spring Boot

Prometheus

Grafana

OpenTelemetry

Micrometer Metrics

KEY TAKEAWAY Use the right Resilience4j component for the right problem, and combine them for maximum resilience -- Lab 32 combines Circuit Breaker, Retry, and Time Limiter on a single method.

TRY IT YOURSELF — EXERCISE 4: PATTERN MAP

Reinforces: assigning each Resilience4j component to a concrete Northstar Account Profile behavior.

PATTERN -> CRM USE (COPY INTO notes/pattern-map.md)

Pattern	CRM Use
Retry	Transient 503 from the Account Profile service.
TimeLimiter	Fail fast if the call exceeds the configured budget (N ms).
CircuitBreaker	Stop calling Account Profile once the failure rate is high.
Fallback	Return a cached or minimal profile for Amina (CUS-1001).

REMAINING STEPS

- **Ravi Row** — Add one example sentence for CUS-1002 (Ravi Singh) describing what the CRM returns while the circuit is open.
- **Decorator Order** — Propose a one-line order, e.g. TimeLimiter -> CircuitBreaker -> Retry -> call.
- **Boundary** — Note: do not apply a circuit breaker to a local in-memory map lookup -- only to real outbound calls.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: [exercises/exercise-04-pattern-map.md](#).

RETRY WITH RESILIENCE4J

Automatically retry failed operations to improve reliability -- configurable via application.yml and applied with a single annotation.

RETRY CONFIGURATION (application.yml)

```
resilience4j:
  retry:
    instances:
      accountProfile:
        max-attempts: 3
        wait-duration: 250ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        retry-exceptions:
          - java.io.IOException
          - c.n.crm.account.TemporaryAccountException
```

Map HTTP 503 to a TemporaryAccountException in the client. Never list business/validation errors as retryable.

RETRY + CIRCUIT BREAKER + TIME LIMITER (SPRING BOOT)

```
@CircuitBreaker(name = "accountProfile",
    fallbackMethod = "fallback")
@Retry(name = "accountProfile")
@TimeLimiter(name = "accountProfile")
public CompletableFuture<AccountSummary> find(
    String customerId) {
    return CompletableFuture.supplyAsync(
        () -> client.fetch(customerId), executor);
}
```

BEST PRACTICE

Retry only transient failures

Never retry 4xx client errors

Use exponential backoff

Combine with Circuit Breaker + Timeout

KEY TAKEAWAY Retry improves the success rate for transient failures, but only for idempotent, safe-to-repeat operations -- never blindly retry writes.

CIRCUIT BREAKER WITH RESILIENCE4J

Resilience4j provides a lightweight Circuit Breaker that monitors calls and short-circuits requests when failures cross a configured threshold.

CIRCUIT BREAKER CONFIGURATION (application.yml)

```
resilience4j:
  circuitbreaker:
    instances:
      accountProfile:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 2
        automatic-transition-from-open-to-half-open-
          enabled: true
```

HOW IT WORKS WITH RESILIENCE4J



CLOSED: calls pass through, failures are recorded. OPEN: calls are rejected immediately (`CallNotPermittedException`); the fallback runs. HALF-OPEN: a limited number of trial requests are allowed through.

COMMON USE CASES

External REST APIs

Third-party services

Database / cache calls

Protecting critical microservices

KEY TAKEAWAY Resilience4j Circuit Breaker helps your system fail fast and recover gracefully. Configure thresholds from real-world metrics, and always pair it with a fallback.

TRY IT YOURSELF — EXERCISE 5: FILL RESILIENCE TODOS (STARTER)

Reinforces: completing the Resilience4j annotation and config blanks for the CRM Account Profile client -- a fill-in-the-blank starter, not a complete solution.

STARTER -- FILL IN THE BLANKS (notes/lab32-todos.md)

```
@CircuitBreaker(name = "____", fallbackMethod = "____")
@Retry(name = "accountProfile")
@TimeLimiter(name = "accountProfile")
public CompletableFuture<AccountProfile> getProfile(String customerId) {
    return _____.fetch(customerId); // remote client
}
private CompletableFuture<AccountProfile> profileFallback(
    String customerId, Throwable t) {
    // TODO: return minimal profile for CUS-1001 / CUS-1002
    return CompletableFuture.completedFuture(____);
}
```

STEPS (IN notes/lab32-todos.md)

Step	What To Do
1 -- Fill Annotations & Fallback	Suggested fills: accountProfile, profileFallback, accountClient, AccountProfile.minimal(customerId).
2 -- Config Blanks	Add YAML TODOs -- failureRateThreshold, waitDurationInOpenState, maxAttempts -- with example numbers you choose.
3 -- Correlation	Add a TODO comment: log lab-request-001 when the fallback fires.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 (STARTER) before continuing: exercises/exercise-05-fill-resilience-todos.md. Fallback must match the return type plus one extra Throwable argument.

BUILDING HIGHLY AVAILABLE SYSTEMS (1 OF 2)

Highly Available (HA) systems are designed to minimize downtime and ensure continuous service delivery, even in the presence of failures.

WHY HIGH AVAILABILITY MATTERS

- **Better User Experience** — users expect services to always be available.
- **Business Continuity** — avoid revenue loss and protect brand reputation.
- **Fault Tolerance** — systems continue to operate even when components fail.
- **Scalability & Growth** — HA architectures support growth without compromising reliability.
- **Compliance & SLAs** — meet regulatory requirements and SLA commitments.

WHAT HIGH AVAILABILITY MEANS

HA is not just about uptime -- it's about minimizing downtime and ensuring fast, seamless recovery.

KEY PRINCIPLES OF HIGH AVAILABILITY



KEY TAKEAWAY Highly available systems eliminate single points of failure, detect issues early, and recover automatically -- keeping the service responsive for users.

BUILDING HIGHLY AVAILABLE SYSTEMS (2 OF 2)

HA architecture patterns, how Resilience4j's components support HA, and the metrics that prove it's actually working.

HIGH AVAILABILITY PATTERNS

Pattern	What It Means
Active-Active	Multiple active instances handle traffic. If one fails, others continue without interruption.
Active-Passive	Standby systems take over only when the active system fails.
Multi-Region	Deploy across regions for disaster tolerance and latency optimization.
Auto-Scaling	Dynamically scale resources based on demand to maintain availability.

RESILIENCE4J'S ROLE IN HA SYSTEMS

Component	HA Contribution
Circuit Breaker	Prevents cascading failures by stopping calls to unhealthy services.
Retry	Automatically retries transient failures to increase success rate.
Time Limiter	Ensures services respond within acceptable timeouts.
Rate Limiter	Protects services from overload by controlling request rate.
Bulkhead	Isolates resources and prevents impact on other services.

AVAILABILITY METRICS TO TRACK

Uptime %

MTTR (Mean Time To Recover)

MTBF (Mean Time Between Failures)

Error Rate

Latency

KEY TAKEAWAY Building highly available systems is about people, process, and technology working together. Design for failure, build with resilience, and deliver uninterrupted value.

RESILIENCE BEST PRACTICES

Resilient systems anticipate failures, absorb the impact, and recover quickly. Follow these best practices across design, isolation, observability, and recovery.

#	Practice	What It Means
1	Design for Failure	Assume components will fail; eliminate single points of failure; set timeouts and limits.
2	Isolate and Contain	Use Bulkhead and Circuit Breaker patterns to stop cascading failures from spreading.
3	Ensure Observability	Centralize logs with correlation IDs; collect key metrics; use distributed tracing and alerts.
4	Build for Recovery	Automate recovery with health checks; plan for failover; back up data; test disaster scenarios.
5	Manage Dependencies	Know, classify, and continuously monitor every external dependency your service relies on.
6	Optimize Performance	Right-size resources, use caching, load test regularly, and scale appropriately.
7	Practice Governance	Define SLAs/SLOs, document runbooks, and learn from every incident to improve continuously.
8	Combine Resilience4j Patterns	Circuit Breaker + Retry + Time Limiter + Rate Limiter + Bulkhead together give layered protection.

PRINCIPLES TO REMEMBER

- Fail Fast
- Resist Failure
- Recover Fast
- Learn & Improve

KEY TAKEAWAY Resilience is not a single feature -- it's a mindset and a set of practices. Design for failure, isolate failures, observe everything, and automate recovery to build truly resilient systems.

TRY IT YOURSELF — EXERCISE 6: LAB 32 READINESS (CAPSTONE)

Reinforces: confirming you can explain the patterns before wiring Resilience4j into the Lab 32 starter -- a pre-lab self-check.

READINESS CHECKLIST (IN `notes/lab32-readiness.md`)

Step	What To Do
1 -- Teach-Back	Explain the Circuit Breaker to a peer in four sentences or fewer -- write the sentences down.
2 -- Evidence Preview	Note the evidence you'll capture in the lab: logs showing the fallback firing for Amina, and a recorded breaker state change.
3 -- No Run Yet	Confirm this pre-lab did not require starting the Account Profile mock or Spring Boot.
4 -- Pass Mark	You pass this check if your pattern map (Exercise 2) and filled TODOs (Exercise 4) both exist in your notes.

READY FOR LAB 32?

Teach-back written

Pattern map (Ex 2) done

TODOs (Ex 4) filled

Mock not started yet

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before continuing: `exercises/exercise-06-lab32-readiness.md`. This is the last checkpoint before Lab 32 -- Resilience4j for CRM Outbound Calls.

LAB OVERVIEW -- RESILIENCE AND FAULT TOLERANCE

Lab 32: Resilience4j for CRM Outbound Calls -- Northstar Account Profile

BUSINESS SCENARIO

- Agents opening Amina Khan (CUS-1001) see account summaries from a separate Account Profile service. When it flaps, the CRM must still show identity/status plus a clear "accounts temporarily unavailable" banner -- never a spinning tab or a lie that a balance updated.
- Leadership freezes the contract: outbound account reads are wrapped with Retry + CircuitBreaker + TimeLimiter; degraded reads return AccountSummary.unavailable; writes are never silently marked successful.
- Depends on Labs 25-31's CRM API -- prefer Lab 31 so Kafka and resilience coexist in the same project.

LEARNING OBJECTIVES

- Reproduce fast, slow, and failing responses with WireMock.
- Configure Resilience4j Retry for transient, safe GET requests.
- Configure a CircuitBreaker; observe CLOSED, OPEN, HALF_OPEN.
- Apply a TimeLimiter that enforces the CRM latency budget.
- Write explicit degraded fallbacks (AccountSummary.unavailable).
- Prevent unsafe retries and false-success fallbacks for writes.

WHAT YOU BUILD

- Copy lab31-crm to lab32-crm; add Resilience4j Boot3 + AOP + Actuator + WireMock; stub 503/slow/OK for CUS-1001; configure Retry/CircuitBreaker/TimeLimiter instance "accountProfile"; annotate AccountProfileService.find with @CircuitBreaker+ @Retry+@TimeLimiter returning CompletableFuture; implement fallback -> AccountSummary.unavailable(customerId); expose Actuator health/metrics/events; write AccountProfileResilienceTest proving OPEN, timeout, and recovery.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan) and CUS-1002 (Ravi Singh) anchor every example; correlation ID lab-request-001 ties requests and logs together. Format: Lecture + Lab -- Theory 1 Hour, Lab 2 Hours, Total 3 Hours.

LAB TASKS AND DELIVERABLES

Resilience4j for CRM Outbound Calls -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch lab31-crm to lab32-crm; add resilience4j-spring-boot3 + spring-boot-starter-aop + actuator.
2	Create deterministic WireMock stubs: 503-then-recover, permanent 503, and a 3000ms slow response for CUS-1001.
3	Configure bounded Retry (accountProfile): max-attempts 3, wait-duration 250ms, exponential backoff x2.
4	Configure CircuitBreaker (accountProfile): count-based window of 10, min calls 5, 50% failure threshold, 10s wait in Open.
5	Configure TimeLimiter (accountProfile): 1500ms timeout, cancel-running-future true; call the client via CompletableFuture.
6	Compose @CircuitBreaker + @Retry + @TimeLimiter on AccountProfileService.find(); call through the Spring bean, never this.
7	Implement fallback() -> AccountSummary.unavailable(customerId); never mark a write as falsely successful.
8	Expose Actuator health/circuitbreakerevents/metrics; write AccountProfileResilienceTest proving OPEN, timeout, and recovery twice.

EXPECTED DELIVERABLES

- Resilience4j Retry + CircuitBreaker + TimeLimiter on account profile reads.
- AccountSummary.unavailable truthful fallback.
- WireMock stubs for 503 / slow / OK on CUS-1001.
- Actuator observation evidence.
- AccountProfileResilienceTest output (OPEN, timeout, recovery); no secrets committed.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY A fallback that implies write success is an honor violation; a circuit breaker configured but never demonstrated OPEN is incomplete; sleep-only tests lose quality marks.

MODULE 32 SUMMARY

In this module, we learned how to design resilient systems, handle transient and persistent failures, and apply Resilience4j patterns to build highly available microservices.

KEY CONCEPTS COVERED



Detect Failures

Identify and handle failures gracefully; failures are inevitable, resilience is a choice.



Retry Smartly

Use fixed and exponential-backoff retry patterns for safe, idempotent, transient failures.



Break the Chain

Stop cascading failures with a Circuit Breaker's Closed / Open / Half-Open states.



Set Timeouts & Fallbacks

Prevent hanging calls with smart timeouts; degrade gracefully with truthful fallbacks.



Leverage Resilience4j

Configure Retry, CircuitBreaker, TimeLimiter, RateLimiter, and Bulkhead via annotations and YAML.



Stay Observable

Monitor health, metrics, and events so resilience patterns are visible, not silent.

MODULE FLOW RECAP

1

Understand Failures

2

Retry + Backoff

3

Circuit Breaker

4

Timeouts + Fallbacks

5

Lab: CRM Outbound Calls

KEY TAKEAWAY Resilience is not a feature -- it's a mindset and a practice. Design for failure, implement resilience patterns, observe continuously, and deliver value without interruption.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of resilience goals, retry, circuit breaker triggers, and rate limiting.

1. What is the primary goal of resilience in distributed systems?

- A. Increase system complexity
- B. Prevent all failures
- C. Continue delivering value despite failures**
- D. Reduce hardware costs

3. When does a Circuit Breaker stop allowing calls to a service (open)?

- A. When the service is slow
- B. When the failure rate exceeds the threshold**
- C. When retry attempts reach the limit
- D. When the system is under high load

2. Which pattern automatically retries a failed call with a backoff strategy?

- A. Circuit Breaker
- B. Retry**
- C. Rate Limiter
- D. Bulkhead

4. Which pattern limits the number of requests sent in a given time window?

- A. Time Limiter
- B. Rate Limiter**
- C. Bulkhead
- D. Fallback

KEY TAKEAWAY Resilience means continuing to deliver value despite failures; Retry re-attempts with backoff; a Circuit Breaker opens on failure-rate threshold; a Rate Limiter caps request volume per window.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on Time Limiter, Bulkhead, the Half-Open state, and safe-to-retry failures.

5. What is the purpose of a Time Limiter?

- A. To retry failed calls
- B. To limit the number of concurrent calls
- C. To restrict the rate of requests
- D. To limit the maximum duration of a call**

7. In a Circuit Breaker, which state allows only a limited number of test calls to check whether a failing service has recovered?

- A. Closed
- B. Open
- C. Half-Open**
- D. Disabled

6. What does the Bulkhead pattern help to do?

- A. Retry failed requests
- B. Isolate resources and prevent failure propagation**
- C. Limit response time
- D. Return fallback responses

8. Per Resilience4j best practice, which of the following should generally NOT be configured as retryable?

- A. A network IOException
- B. A 503 Service Unavailable response
- C. A validation-driven 4xx client error**
- D. A socket timeout

KEY TAKEAWAY Time Limiter bounds call duration; Bulkhead isolates resources; Half-Open safely probes recovery with limited test calls; 4xx client/validation errors should never be retried.

MODULE 32 COMPLETE

Resilience and Fault Tolerance Patterns

You can now design for failure, apply Retry, Circuit Breaker, Timeout, and Fallback patterns with Resilience4j, and build highly available microservices that keep delivering value.